Assignment 2 Report

Machine Learning (COMS3007A)

School of Computer Science & Applied Mathematics
University of the Witwatersrand

Nkosinathi Khumalo, Mpho Murashiwa, Tjeding Makubung, Kamogelo Mosehle
2805324, 2540206, 2807685, 2811548

May 22, 2026



1 Exploratory Data Analysis and Imputation

1.1 Cleaning, Missing Values, Duplicates, and Outliers

Only `Signal_F` had missing values: 709 in training and 506 in test. We used forward/backward fill within each `Sample_ID` because readings are time ordered, then used the median as a fallback. This kept all 100-step samples complete. No duplicate sensor rows or duplicate label mappings were found. Outliers were not removed because large signal changes may represent real movement.

1.2 EDA Findings

The classes were not perfectly balanced, so Macro-F1 was used to give each class equal importance. Signal summaries showed different ranges, so scaling was needed. Time plots showed changes across the 100 steps, confirming that temporal order was useful.

2 Dimensionality Reduction and Feature Engineering

2.1 Feature Engineering and PCA

Each 100-step sample was converted into one row with 508 features, including statistics, RMS, energy, differences, slopes, segment summaries, FFT features, and signal correlations. These captured signal strength, temporal behaviour, frequency patterns, and relationships between anonymised signals. PCA reduced the 508 features to 101 components while keeping 95% variance, showing redundancy in the engineered features.

3 Local Validation Strategy

3.1 Validation Split

We used an 80/20 stratified split, giving 2234 training samples and 559 validation samples. Stratification kept class proportions similar, making Macro-F1 more reliable.

3.2 Sequence Preparation

For neural models, the data was reshaped into $(2793, 100, 14)$. The split gave $(2234, 100, 14)$ for training and $(559, 100, 14)$ for validation. Signals were scaled with `StandardScaler`, and labels were one-hot encoded into six classes.

4 Model Architecture and Hyperparameters

4.1 Baseline Models

Logistic Regression was used as a simple engineered-feature baseline and achieved 0.720 Macro-F1. Extra Trees improved this to 0.802, showing that non-linear models used the engineered features better. Important features included correlation, RMS, absolute mean, frequency, and difference-based features.

4.2 Sequence Models

We then moved to sequence models because each sample is ordered over 100 time steps. LSTM achieved 0.866 by learning temporal changes. CNN-LSTM improved on this because Conv1D learns short local signal patterns while LSTM learns how they change over the full sequence. The CNN-LSTM ensemble performed best by averaging multiple CNN-LSTM predictions.

4.3 Hyperparameter Tuning

A small CNN-LSTM hyperparameter test was performed using validation Macro-F1. The best tuned configuration achieved 0.871, but the CNN-LSTM ensemble still performed better with 0.904, so the ensemble was selected.

4.4 Final Model

The CNN-LSTM base model used two Conv1D layers with 64 filters, kernel sizes 5 and 3, batch normalisation, max pooling, dropout, an LSTM layer with 64 units, a dense layer with 32 units, and a six-class softmax output. It was trained with Adam, categorical cross-entropy, batch size 32, up to 60 epochs, and early stopping. The final selected model was a CNN-LSTM ensemble, created by averaging predictions from multiple CNN-LSTM runs.

5 Expected Performance

The CNN-LSTM ensemble achieved the best validation Macro-F1 of 0.904 and was selected for the final submission.

Model	Macro-F1
Logistic Regression	0.720
Extra Trees	0.802
LSTM	0.866
CNN-LSTM	0.876
CNN-LSTM Ensemble	0.904

Table 1: Validation Macro-F1 scores from the final clean run.

References